

CSE 256 | Lecture 17: Retrieval Augmented Generation (RAG)

Ndapa Nakashole

March 05, 2026

Administrative matters

- ▶ **PA2: grades**
- ▶ **Two more lectures left!**
 - ◉ RAG (today)
 - ◉ Last lecture (March 10)
 - ◉ March 12: no class (come to CSE 4108, my office: AMA, including PA3!)

Today

- 1 Why Parametric Memory Is Not Enough
- 2 RAG Overview
- 3 Neural Retrieval Methods
 - Bi-Encoder (+ Contrastive Learning)
 - Cross-Encoder
 - ColBERT
- 4 When and How to Retrieve
- 5 Failure Modes and Open Questions

Two Types of Memory in Neural Models

► Parametric memory

- ◉ Knowledge stored in weights
- ◉ Learned during pretraining

► Non-parametric memory

- ◉ External database
- ◉ Retrieved at inference time

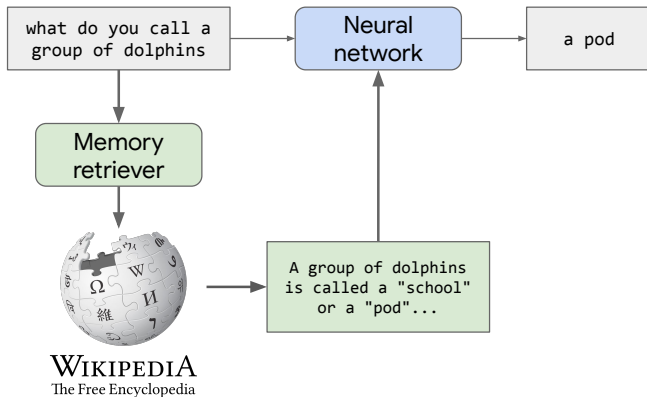
RAG = Parametric Generator + Non-Parametric Memory

Wish List

- ▶ Fast and modular **knowledge editing**
 - ⊙ Robustly update the model N times without breaking its behavior on other tasks
- ▶ **Attribution and interpretability**
 - ⊙ Trace a model's knowledge back to a particular document / training example
- ▶ **Efficient scaling**
 - ⊙ Increase the model's memory size by $10x$ without paying $10x$ more compute.

RAG addresses all three to some extent

What is a Retrieval Augmented Generation (RAG) model?



- ▶ Step 1: Retrieve relevant documents $d_1, \dots, d_k$ given query x
- ▶ Step 2: Condition the generator on $(x, d_1, \dots, d_k)$
- ▶ Step 3: Generate answer y

RAG is not Prompt Engineering

- ▶ Prompt engineering: manually insert context
- ▶ RAG: learned retrieval + generation pipeline

Key difference:

- ▶ Retrieval is dynamic and query-dependent
- ▶ Can scale to millions of documents

Probabilistic View of RAG

Instead of modeling:

$$p(y \mid x)$$

RAG models:

$$p(y \mid x) = \sum_d p(y \mid x, d) p(d \mid x)$$

- ▶ $p(d \mid x) \rightarrow$ Retriever
- ▶ $p(y \mid x, d) \rightarrow$ Generator

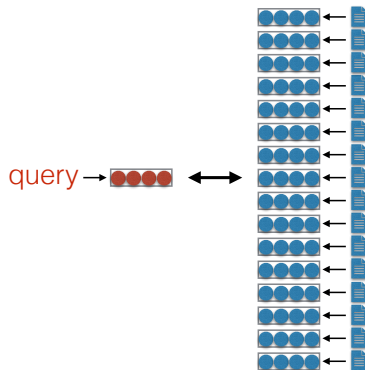
Modeling the Retriever

$$p(y \mid x) = \sum_d p(y \mid x, d) p(d \mid x)$$

How to model $p(d \mid x)$?

Dense Retrieval

- ▶ **Problem with Sparse Retrieval**
(e.g., TF-IDF, BM25):
 - Relies on token overlap
 - Suffers from **vocabulary mismatch**
- ▶ **Dense Retrieval:**
 - Map query x and document d to vectors
 - Measure **semantic similarity** in embedding space
- ▶ **Two Approaches:**
 - Off-the-shelf embeddings (e.g., BERT)
 - Fine-tuned retrieval models



Dense Retrieval: How to Score (x, d) ?

- ▶ Want to model $\text{score}(x, d)$, but there are multiple design choices
 - ◉ Encode query and document **independently**
 - ◉ Encode them **jointly**
 - ◉ ... something in between ?

Tradeoff in modeling query–document interactions

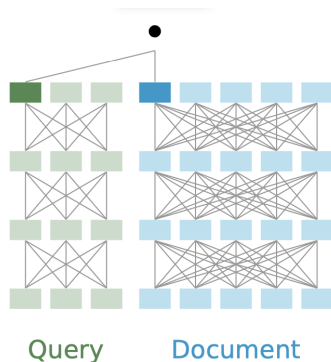
Expressiveness

- ▶ **Fine-grained** query–document interactions
- ▶ vs. **Single-vector** representations

Efficiency

- ▶ **Pre-computation** of document representations
- ▶ **Low query latency** for real-time systems

Bi-Encoder



- ▶ Encode query x and document d **independently**
- ▶ Map each to a **single-vector** representation in $\mathbb{R}^k$
- ▶ Compute similarity:

$$\text{score}(x, d) = \langle f(x), g(d) \rangle$$

Advantages of Bi-Encoder:

- ▶ Document embeddings can be **precomputed**
- ▶ Retrieval reduces to **fast nearest-neighbor search**

Nearest Neighbor Search in Vector Space

Given a query vector q , we want the most similar document (s):

$$\arg \max_i q^\top d_i$$

Brute force search:

- ▶ Compute similarity with **every document vector**
- ▶ Complexity: $O(Nd)$
 - ◉ N is the number of documents
 - ◉ d is the embedding dimension
- ▶ Works for thousands of documents, **too slow for millions or billions**

Want: **Approximate Nearest Neighbor (ANN)** search.

Fast Approximate Search - FAISS library

FAISS (Facebook AI Similarity Search) accelerates nearest neighbor search using **vector indexing**.

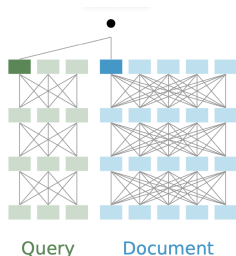
Vector Indexing

- ▶ Use K-Means to partition the vector space into k cells
- ▶ At query time, find the nearest cell center and **only search vectors inside that cell**.
- ▶ Reduces N to N/k .

Usually combined with **Quantization** for further speedup.

Fast retrieval with small accuracy loss!

Bi-Encoder Example: Dense Passage Retrieval (DPR)



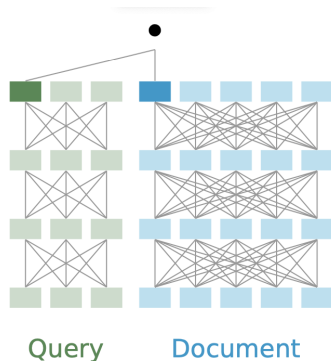
- ▶ Encode query and passage into **768-dim vectors**

- ▶ Score using **dot product**:

$$s(x, d) = \langle f(x), g(d) \rangle$$

- ▶ Train with **contrastive loss** over:
 - ◉ One positive passage (per query)
 - ◉ Hard negatives (e.g., BM25 top-k)
 - ◉ In-batch negatives

Bi-Encoder Training



- 1 Training examples:

$$(q_i, d_i^+, \{d_{i,j}^-\})$$

- 2 Similarity score between query and document:

$$s(q, d) = \langle f(q), g(d) \rangle$$

- 3 Loss: negative log-likelihood of the positive passage

$$\mathcal{L} = -\log \frac{\exp(s(q_i, d_i^+))}{\exp(s(q_i, d_i^+)) + \sum_j \exp(s(q_i, d_{i,j}^-))}$$

Contrastive Training

Contrastive Loss

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(q, d^+))}{\exp(\text{sim}(q, d^+)) + \sum_{j=1}^n \exp(\text{sim}(q, d_j^-))}$$

Maximize Agreement

Pull the query and the **positive** example together in the embedding space.

$\leftrightarrow$

Minimize Agreement

Push the query away from all **negative** examples (noise) in the batch.

- ▶ **Key Idea:** We don't need absolute scores; we need **relative** ranking.
- ▶ **Generalization:** This same objective powers CLIP (Images/Text), SimCLR (Vision), and ... many more!

Bi-Encoder: Strengths and Weaknesses

► Strengths

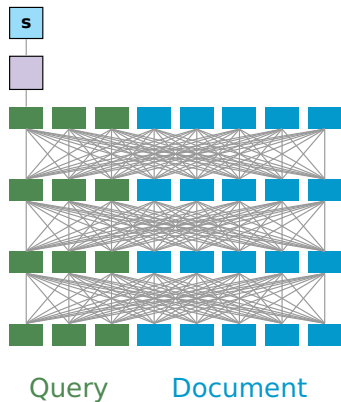
- ◉ **Fast** retrieval(pre-computable document embeddings)
- ◉ **Scales** to millions of documents
- ◉ Works well with **contrastive learning**

► Weakness

- ◉ No query–document interaction during encoding
- ◉ Limited ability to model fine-grained token-level matching

How can we allow richer interactions?

Cross-Encoder



- ▶ **Concatenate** query and document
- ▶ **Encode jointly** with full self-attention
- ▶ Produce a single relevance score

Scoring $s(q, d) = f([q; d])$

Training

- ▶ Same contrastive / softmax loss
- ▶ But representations depend on full interaction

Strength: Rich token-level interactions **Weakness:** Cannot

precompute documents → expensive at retrieval time

Can We Get the Best of Both Worlds?

Bi-Encoder

Fast / Scalable

Coarse-grained interaction

vs.

Cross-Encoder

Prohibitively Slow

Rich / Token-level interaction

Can we keep **precomputation** and allow **fine-grained interactions**?

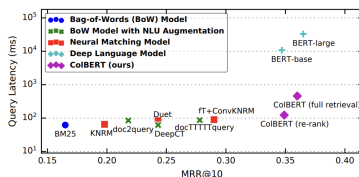
ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT

Omar Khattab
Stanford University
okhattab@stanford.edu

Matei Zaharia
Stanford University
matei@cs.stanford.edu

ABSTRACT

Recent progress in Natural Language Understanding (NLU) is driving fast-paced advances in Information Retrieval (IR), largely owed to fine-tuning deep language models (LMs) for document ranking. While remarkably effective, the ranking models based on these LMs increase computational cost by orders of magnitude over prior approaches, particularly as they must feed each query-document pair through a massive neural network to compute a single relevance score. To tackle this, we present ColBERT, a novel ranking model that adapts deep LMs (in particular, BERT) for efficient retrieval. ColBERT introduces a *late interaction* architecture that indepen-



ColBERT: Late Interaction Retrieval

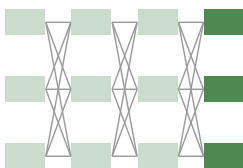
- ▶ Each token is embedded separately
- ▶ Document is represented as a **matrix**, not a single vector:

$$\mathbf{D} \in \mathbb{R}^{L \times d}$$

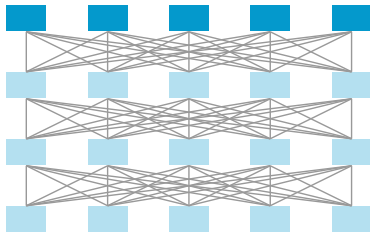
- ▶ Query tokens interact with document tokens **after encoding**

$$\text{MaxSim} = .97 + .84 + .85$$

Query



.55	.61	.72	.76	.85
.64	.84	.71	.60	.80
.82	.97	.82	.72	.90



Document

ColBERT Scoring: Late Interaction

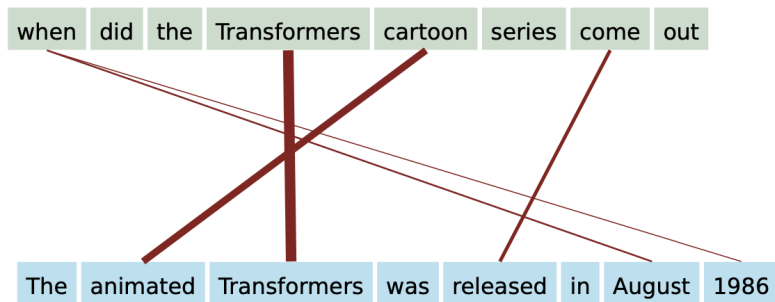
Each query token q_i finds its best match in the document:

$$\text{score}(q, d) = \sum_{i \in q} \max_{j \in d} \langle q_i, d_j \rangle$$

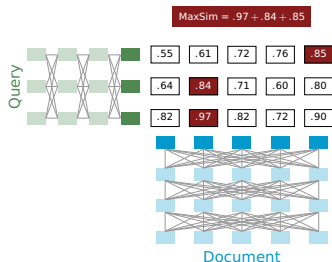
- ▶ Fine-grained token matching
- ▶ But document token embeddings are precomputed

Interaction without re-encoding documents

Soft Alignment with ColBERT



ColBERT Training



Training examples

$$\langle q_i, d_i^+, \{d_{i,j}^-\} \rangle$$

Scoring via MaxSim

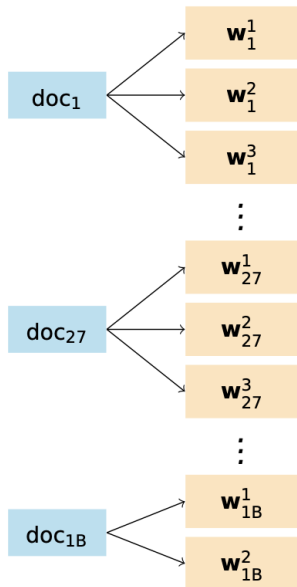
$$\text{score}(q, d) = \sum_{i=1}^L \max_j \langle q_i, d_j \rangle$$

Loss

- ▶ Same softmax / contrastive loss
- ▶ Replace dot product with MaxSim

Fine-grained token interaction
while keeping document embeddings precomputed.

ColBERT as a Re-Ranker



Two-Stage Retrieval Pipeline

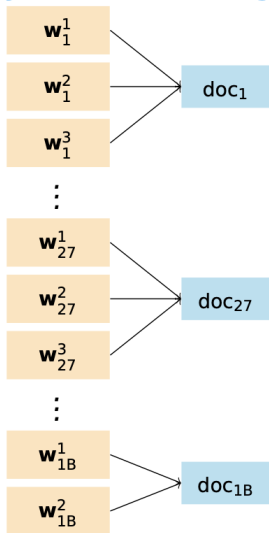
Given a query $q = [w^1, \dots, w^M]$:

- 1 Retrieve top- K documents using a fast model (e.g., BM25 or a Bi-Encoder)
- 2 Re-rank those K candidates using ColBERT

Why?

- ▶ Full ColBERT scoring over entire corpus is expensive
- ▶ Re-ranking keeps efficiency while improving precision

ColBERT Beyond Re-Ranking: Direct Retrieval with Late Interaction



Direct Retrieval with Late Interaction

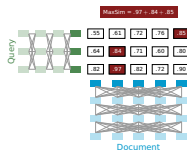
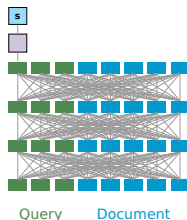
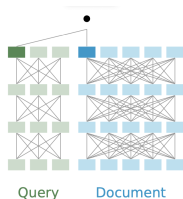
Given query token embeddings $[q_1, \dots, q_M]$:

- 1 For each query token q_i , retrieve the top- p nearest document token embeddings.
- 2 Aggregate matching documents using the MaxSim scoring rule.

Key Idea:

- Build an index over document token embeddings
- Retrieve documents via token-level

Summary: Dense Retrieval Architectures



Bi-Encoder

- ✓ Precomputed document vectors
- ✗ Limited query-document interaction

Cross-Encoder

- ✓ Full token-level interaction
- ✗ No document precomputation (slow at scale)

ColBERT (Late Interaction)

- ✓ Precomputed token embeddings
- ✓ Fine-grained late interaction

Tradeoff:

Expressiveness

vs.

Scalability

When Should a Model Retrieve?

RAG is not just *what* to retrieve, it is also *when* in the generation process to retrieve.

When Do We Retrieve?

- 1 Retrieve **Once** (at the start)
 - Most common approach
 - Works well for short answers
 - Limiting for long-form generation
- 2 Retrieve **Periodically**
 - Every k tokens (fixed schedule)
- 3 Retrieve **Adaptively**
 - Model decides when to retrieve
 - Example: **Toolformer**

Toolformer: Language Models Can Teach Themselves to Use Tools

**Timo Schick Jane Dwivedi-Yu Roberto Dessì[†] Roberta Raileanu
Maria Lomeli Luke Zettlemoyer Nicola Cancedda Thomas Scialom**

Meta AI Research [†]Universitat Pompeu Fabra

Abstract

Language models (LMs) exhibit remarkable abilities to solve new tasks from just a few examples or textual instructions, especially at scale. They also, paradoxically, struggle with basic functionality, such as arithmetic or factual lookup, where much simpler and smaller models excel. In this paper, we show that LMs can teach themselves to *use external tools*

The New England Journal of Medicine is a registered trademark of [QA("Who is the publisher of The New England Journal of Medicine?") → Massachusetts Medical Society] the MMS.

Out of 1400 participants, 400 (or [Calculator(400 / 1400) → 0.29] 29%) passed the test.

The name derives from "la tortuga", the Spanish word for

Language Models Can Teach Themselves to Use Tools

- ▶ Model generates special tokens that trigger tool calls
- ▶ Retrieval is invoked only when useful

The New England Journal of Medicine is a registered trademark of [QA("Who is the publisher of The New England Journal of Medicine?") → Massachusetts Medical Society] the MMS.

Out of 1400 participants, 400 (or [Calculator(400 / 1400) → 0.29] 29%) passed the test.

The name derives from "la tortuga", the Spanish word for [MT("tortuga") → turtle] turtle.

The Brown Act is California's law [WikiSearch("Brown Act") → The Ralph M. Brown Act is an act of the

Toolformer Training

Given plain text dataset $\mathcal{C}$, construct augmented dataset $\mathcal{C}^*$ with API calls.

Step 1: Propose API Calls

- ▶ Few-shot prompt LM to insert potential tool calls

Step 2: Filter

- ▶ Execute API calls
- ▶ Keep calls that reduce next-token loss

Step 3: Finetune

- ▶ Train LM on filtered augmented dataset $\mathcal{C}^*$

Toolformer: use LM to get large number of potential API calls

Your task is to add calls to a Question Answering API to a piece of text. The questions should help you get information required to complete the text. You can call the API by writing "[QA(question)]" where "question" is the question you want to ask. Here are some examples of API calls:

Input: Joe Biden was born in Scranton, Pennsylvania.

Output: Joe Biden was born in [QA("Where was Joe Biden born?")] Scranton, [QA("In which state is Scranton?")] Pennsylvania.

Input: Coca-Cola, or Coke, is a carbonated soft drink manufactured by the Coca-Cola Company.

Output: Coca-Cola, or [QA("What other name is Coca-Cola known by?")] Coke, is a carbonated soft drink manufactured by [QA("Who manufactures Coca-Cola?")] the Coca-Cola Company.

Input: x

Output:

Failure Modes of RAG

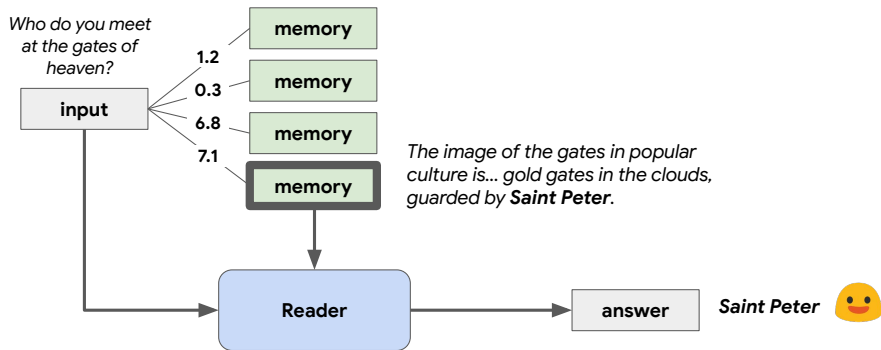
What Can Go Wrong with RAG?

When we add retrieval to a language model, what new failure modes emerge?

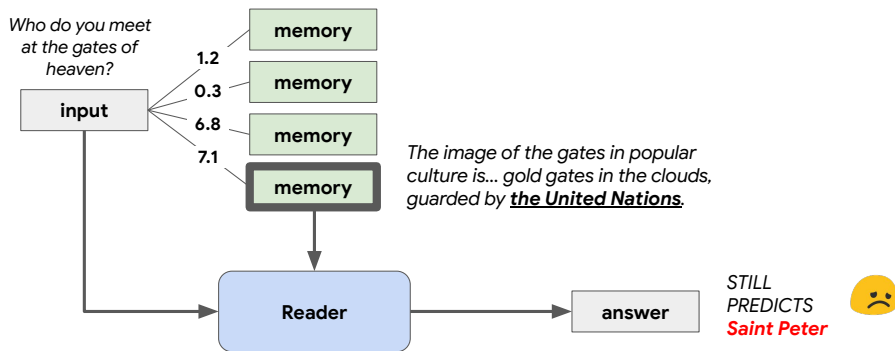
- ▶ **Underutilization** Model ignores retrieved documents.
- ▶ **Overreliance** Model copies or trusts retrieved text too much.

The challenge: balancing parametric and retrieved knowledge.

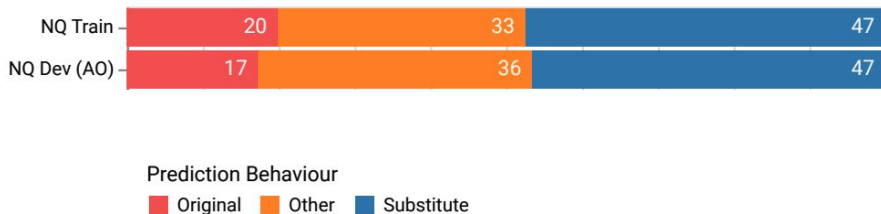
Underutilization of memories (Longpre et al, 2021)



Underutilization of memories (Longpre et al, 2021)



How serious is this problem?



- ▶ Substitute Natural Questions (NQ) dataset answer with incorrect answer in retrieved memory where model originally got it right without retrieval.
- ▶ Evaluated on a subset of examples that the original model got right.
- ▶ Only in $\approx 50\%$ of examples, the model changes its answer to match the retrieved memory.

Failure Mode 1: Underutilization of memories, what is happening?

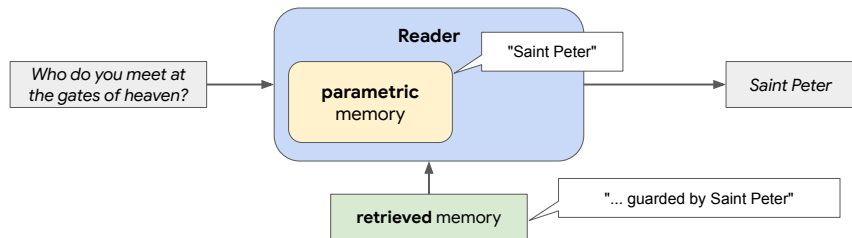
Retrieved documents are relevant, but the model ignores them.

Possible cause:

- ▶ **Training mismatch:** it learned to store the answer in its parametric memory, rather than learning to read the retrieved memory.

Result: Hallucination despite correct evidence.

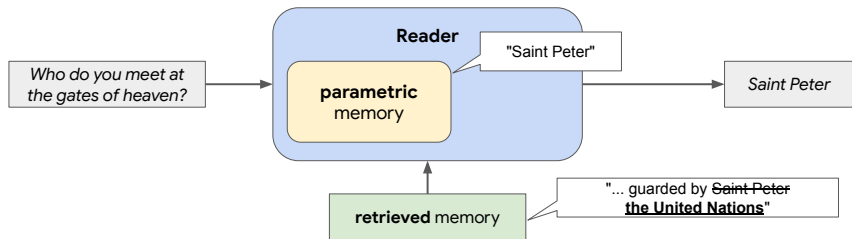
Why Standard RAG Training May Fail



- ▶ If both parametric and retrieved knowledge are correct,
- ▶ The model can ignore retrieval without penalty.

We must create training situations where retrieval matters.

The Counterfactual Trick: Create a Conflict

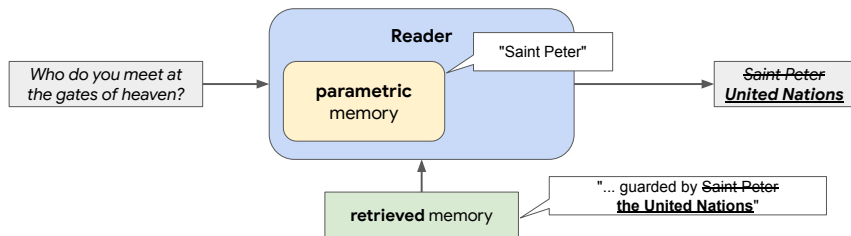


- ▶ Modify retrieved memory
- ▶ Make it contradict parametric memory

Parametric: wrong

Retrieved: correct

Shift the Learning Signal

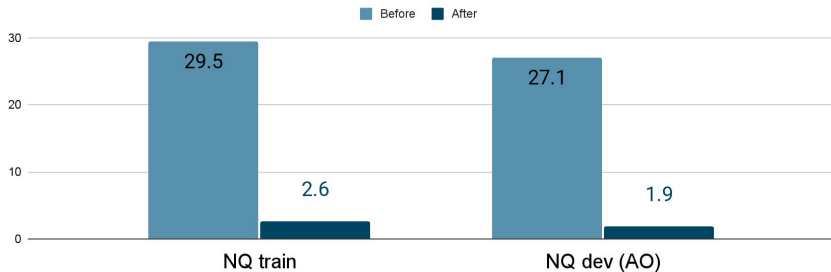


- ▶ Change the gold answer to match retrieved memory
- ▶ The model is penalized for relying on parametric memory

Model learns: retrieval overrides memorization.

Does it work?

% of the time where model incorrectly reverts to original answer.
(ignoring cases where it produces neither old nor new answer)



Beyond RAG Fine-Tuning: Retrieval at Pretraining Scale

So far:

- ▶ Retrieval added during fine-tuning
- ▶ Retrieval used at inference

What if retrieval is built into language modeling from the start?

RETRO : Retrieval-Enhanced Language Modeling

ICML 2022, DeepMind



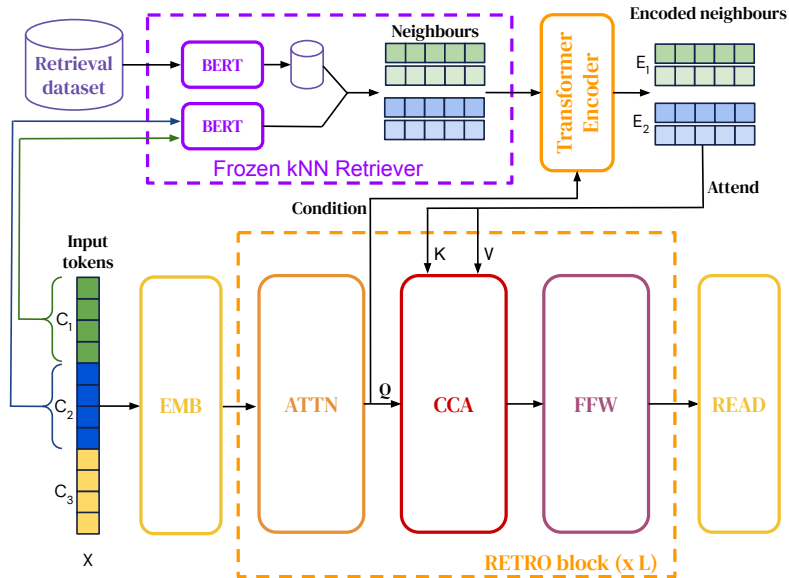
Improving language models by retrieving from trillions of tokens

Sebastian Borgeaud[†], Arthur Mensch[†], Jordan Hoffmann[†], Trevor Cai, Eliza Rutherford, Katie Millican, George van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego de Las Casas, Aurelia Guy, Jacob Menick, Roman Ring, Tom Hennigan, Saffron Huang, Loren Maggiore, Chris Jones, Albin Cassirer, Andy Brock, Michela Paganini, Geoffrey Irving, Oriol Vinyals, Simon Osindero, Karen Simonyan, Jack W. Rae[‡], Erich Elsen[‡] and Laurent Sifre^{†,‡}

All authors from DeepMind, [†]Equal contributions, [‡]Equal senior authorship

- ▶ Trained with a **2 trillion token retrieval database**
- ▶ Fuses retrieved content via cross-attention
- ▶ Achieves GPT-3-level performance
- ▶ With **25× fewer parameters** (7B vs 175B)

Memory fusion with Cross-Attention



RAG Alternatives and Open Questions

- ▶ **Confidence-Based Gating** : Retrieve (or trust parametric memory) based on model uncertainty.
- ▶ **Post-hoc Verification**: Generate first → then verify against retrieved evidence.
- ▶ **Reinforcement Learning-Based Retrieval**.: Treats retrieval as an action. Reward = downstream generation quality.
- ▶ **Attribution Supervision**: Train the model to cite supporting spans explicitly.

Is grounding solved?

No. Balancing parametric and retrieved knowledge remains an open research problem.

Main Takeaways

- ▶ A retriever is a function: $f(\text{input}, \text{memory}) \rightarrow \text{score}$
- ▶ **Training Retrievers**
 - ◉ **Supervised:** use positive and negative examples to learn ranking
 - ◉ **Unsupervised:** synthetic data generation (e.g., Contriever). But unsupervised methods still lag behind strong supervised retrievers
- ▶ **Retrieval is easy. Using retrieved knowledge is hard.**
 - ◉ When to retrieve
 - ◉ How to balance retrieved vs parametric knowledge

That's all for today

Next week: **Last Lecture on Tuesday, March
10th!**